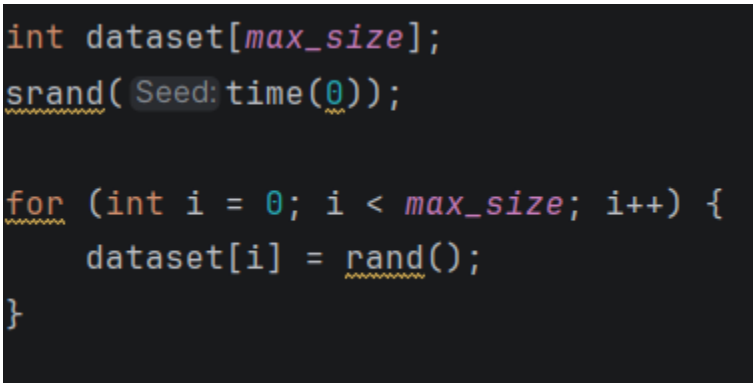
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	8/18/26	**DATE SUBMITTED:**	8/27/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6 **Searching Techniques**

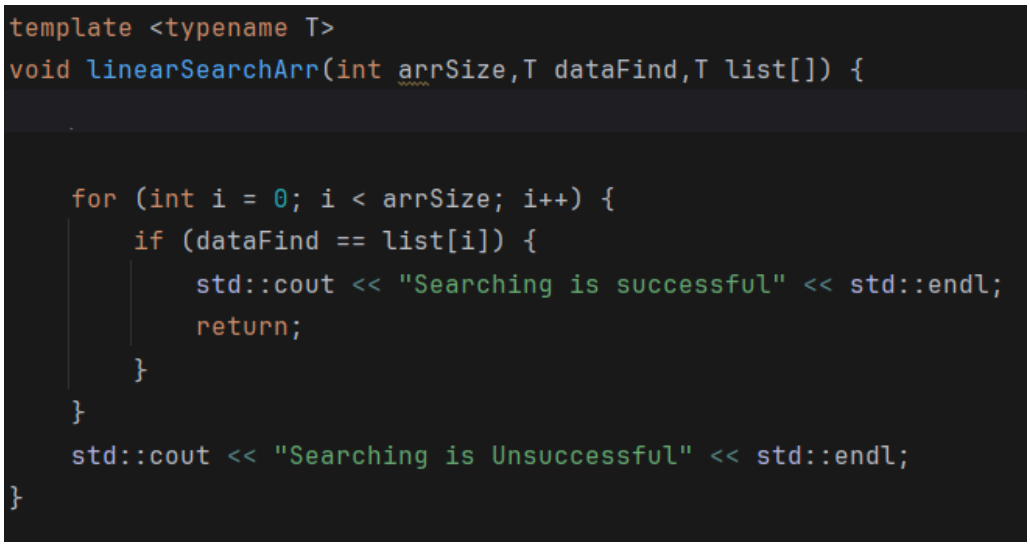
1. PROCEDURE OUTPUT

Screenshot	
Observation	I have observed that the values that were created was random and it will same amount of numbers to the max_size.

```
int dataset[max_size];
srand( Seed: time(0));

for (int i = 0; i < max_size; i++) {
    dataset[i] = rand();
}
```

Table 6-1. Data Generated and Observations.

Code	

```
template <typename T>
void linearSearchArr(int arrSize,T dataFind,T list[]) {


    for (int i = 0; i < arrSize; i++) {
        if (dataFind == list[i]) {
            std::cout << "Searching is successful" << std::endl;
            return;
        }
    }
    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

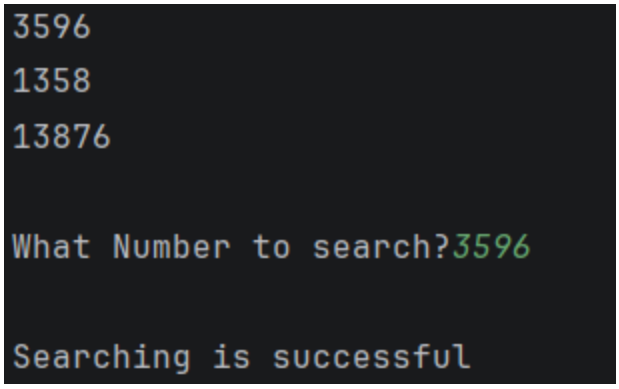
Output	
Observation	Based on my observation, the program checks for the value from the user input and it will print searching successfully if the value is inside the array and will print not successful if it's not in the array.

Table 6-2a. Linear Search for Arrays

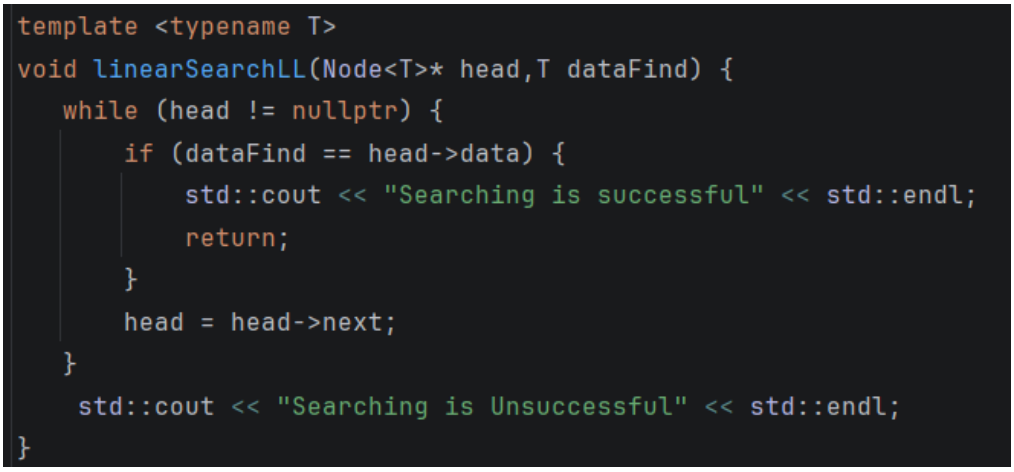
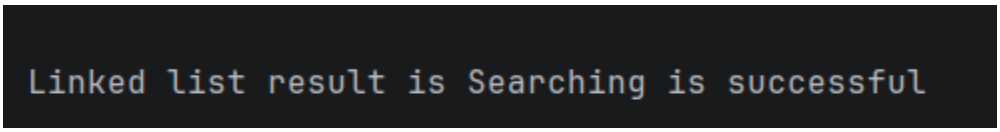
Code	
Output	
Observation	Same with the array based, linked list binary search will search for the value one by one until it reaches value.

Table 6-2b. Linear Search for Linked List

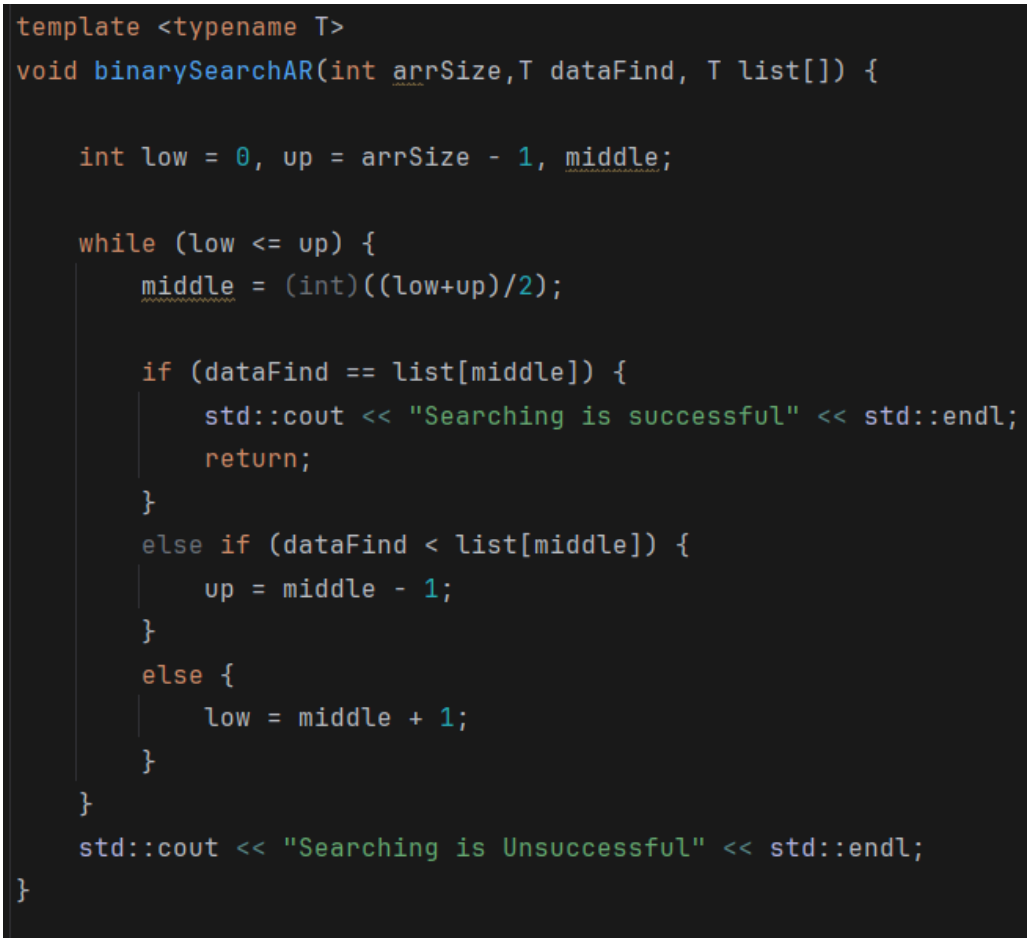
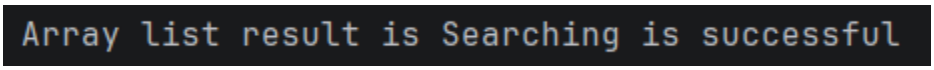
Code	
Output	
Observation	The array needs to be sorted out first in order to implement binary search for arrays. After sorting the array, the binary search will now search for the value from the user input and will start to compare the values.

Table 6-3a. Binary Search for Arrays

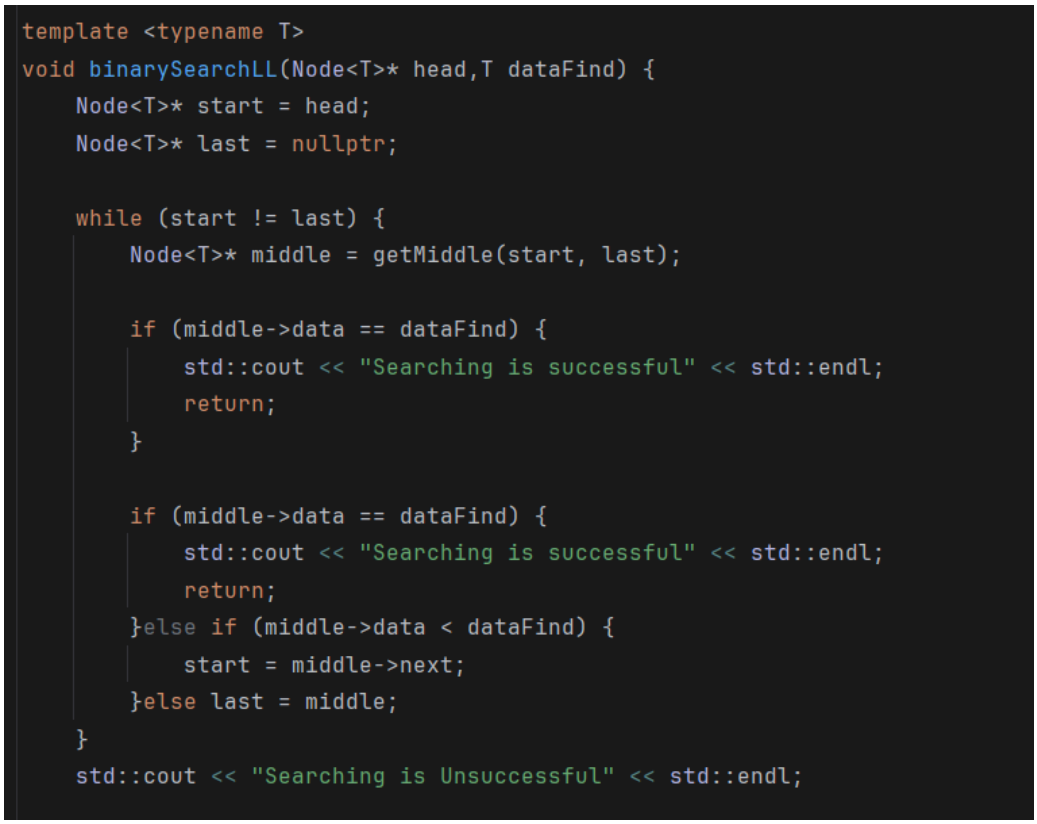
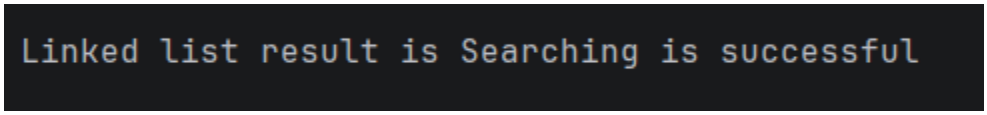
Code	
Output	
Observation	Unlike in array binary search, linked lists do not have to be sorted in order to work. The algorithm will search for the value by comparing.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

```
int main() {
    int problemList[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int listSize = 10;
    int value = 18;

    std::cout << "Problem 1" << std::endl;
    std::cout<< "Linear Search" <<std::endl;
    linearSearchArr(listSize,  dataFind: value, problemList);
    std::cout << std::endl;

    Node<int>* head = newNode( newData: 15);
    head->next = newNode( newData: 18);
    head->next->next = newNode( newData: 2);
    head->next->next->next = newNode( newData: 19);
    head->next->next->next->next = newNode( newData: 18);
    head->next->next->next->next->next = newNode( newData: 0);
    head->next->next->next->next->next->next = newNode( newData: 8);
    head->next->next->next->next->next->next->next = newNode( newData: 14);
    head->next->next->next->next->next->next->next->next = newNode( newData: 19);
    head->next->next->next->next->next->next->next->next->next = newNode( newData: 14);

    std::cout << "Binary Search" << std::endl;
    linearSearchLL(head, value);

    return 0;
}
```

```
Problem 1
Linear Search
Searching is successful. Total Comparison: 3

Binary Search
Searching is successful
```

```cpp
template <typename T>
void linearSearchArr(int arrSize,T dataFind, const T list[]) {
    int num = 0;
    num++;

    for (int i = 0; i < arrSize; i++) {
        num++;
        if (dataFind == list[i]) {
            std::cout << "Searching is successful. Total Comparison: " << num << std::endl;
            return;
        }
    }
    std::cout << "Searching is Unsuccessful. Total Comparison: " << num << std::endl;
}
```

```cpp
template <typename T>
void binarySearchAR(int arrSize, T dataFind, const T list[]) {
    int low = 0, up = arrSize - 1, middle;
    int num = 0;

    while (low <= up) {
        num++;
        middle = low + (up - low) / 2;

        if (dataFind == list[middle]) {
            std::cout << "Searching is successful. Total Comparison: " << num << std::endl;
            return;
        }
        else if (dataFind < list[middle]) {
            up = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "Searching is Unsuccessful. Total comparisons: " << num << std::endl;
}
```

Problem 2:

```cpp
template <typename T>
void linearSearchArr(int arrSize,T dataFind, const T list[]) {
    int num = 0;
    num++;

    for (int i = 0; i < arrSize; i++) {
        num++;
        if (dataFind == list[i]) {
            std::cout<<num << std::endl;
            return;
        }
    }
    std::cout << num << std::endl;
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T>* head,T dataFind) {
    int num = 0;

   while (head != nullptr) {
        num++;
        if (dataFind == head->data) {
            std::cout << num << std::endl;
            return;
        }
        head = head->next;
   }
    std::cout << num << std::endl;
}
```

```
Linear Search
3

Binary Search
2
```

```cpp
int main() {
    int problemList[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int listSize = 10;
    int value = 18;

    std::cout<< "Linear Search" <<std::endl;
    linearSearchArr(listSize, dataFind: value, problemList);
    std::cout << std::endl;

    Node<int>* head = newNode( newData: 15);
    head->next = newNode( newData: 18);
    head->next->next = newNode( newData: 2);
    head->next->next->next = newNode( newData: 19);
    head->next->next->next->next = newNode( newData: 18);
    head->next->next->next->next->next = newNode( newData: 0);
    head->next->next->next->next->next->next = newNode( newData: 8);
    head->next->next->next->next->next->next->next = newNode( newData: 14);
    head->next->next->next->next->next->next->next->next = newNode( newData: 19);
    head->next->next->next->next->next->next->next->next->next = newNode( newData: 14);

    std::cout << "Binary Search" << std::endl;
    linearSearchLL(head, value);



    return 0;
```

Problem 3:

```cpp
template <typename T>
void binarySearchDiagram(int arrSize, T dataFind, const T list[]) {
    int low = 0, up = arrSize - 1, middle;
    int num = 1;

    while (low <= up) {
        middle = low + (up - low) / 2;

        std::cout << "Iteration " << num++ << ": "
                  << "low=" << low << ", up=" << up
                  << ", middle=" << middle << ", Value=" << list[middle] << "\n";

        if (dataFind == list[middle]) {
            std::cout << "Searching is successful\n";
            return;
        }
        else if (dataFind < list[middle]) {
            up = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "Searching is Unsuccessful\n";
}
```

```
    int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int listSize2 = 10;
    int value2 = 8;

    std::cout<< "Linear Search" <<std::endl;
    linearSearchArr(listSize,  dataFind: value, problemList);
    std::cout << std::endl;

    Node<int>* head = newNode( newData: 15);
    head->next = newNode( newData: 18);
    head->next->next = newNode( newData: 2);
    head->next->next->next = newNode( newData: 19);
    head->next->next->next->next = newNode( newData: 18);
    head->next->next->next->next->next = newNode( newData: 0);
    head->next->next->next->next->next->next = newNode( newData: 8);
    head->next->next->next->next->next->next->next = newNode( newData: 14);
    head->next->next->next->next->next->next->next->next = newNode( newData: 19);
    head->next->next->next->next->next->next->next->next->next = newNode( newData: 14);

    std::cout << "Binary Search" << std::endl;
    linearSearchLL(head, value);

    std::cout << std::endl;
    binarySearchDiagram(listSize,  dataFind: value2, sortedList);
```

```
Iteration 1: low=0, up=9, middle=4, Value=11
Iteration 2: low=0, up=3, middle=1, Value=5
Iteration 3: low=2, up=3, middle=2, Value=6
Iteration 4: low=3, up=3, middle=3, Value=8
Searching is successful
```

Problem 4:

```cpp
template <typename T>
void binarySearchRecursive(const T list[], T dataFind, int low, int up) {
    if (low > up) {
        std::cout << "Searching is Unsuccessful\n";
        return;
    }

    int middle = low + (up - low) / 2;

    if (dataFind == list[middle]) {
        std::cout << "Searching is successful at  " << middle << "\n";
        return;
    }
    else if (dataFind < list[middle]) {
        binarySearchRecursive(list, dataFind, low, up: middle - 1);
    }
    else {
        binarySearchRecursive(list, dataFind, low: middle + 1, up);
    }
}
```

```
    int initialLow = 0;
    int initialUp = listSize - 1;

    std::cout<< "Linear Search" <<std::endl;
    linearSearchArr(listSize, dataFind: value, problemList);
    std::cout << std::endl;

    Node<int>* head = newNode( newData: 15);
    head->next = newNode( newData: 18);
    head->next->next = newNode( newData: 2);
    head->next->next->next = newNode( newData: 19);
    head->next->next->next->next = newNode( newData: 18);
    head->next->next->next->next->next = newNode( newData: 0);
    head->next->next->next->next->next->next = newNode( newData: 8);
    head->next->next->next->next->next->next->next = newNode( newData: 14);
    head->next->next->next->next->next->next->next->next = newNode( newData: 19);
    head->next->next->next->next->next->next->next->next->next = newNode( newData: 14);

    std::cout << "Binary Search" << std::endl;
    linearSearchLL(head, value);

    std::cout << std::endl;
    binarySearchDiagram(listSize, dataFind: value2, sortedList);
    binarySearchRecursive(sortedList, dataFind: value2, initialLow, initialUp);
```

```
Searching is successful at  3
```

Conclusion:
During the supplementary, I have created multiple codes where they will return the count of how many times they have attempted to compare the value from the value needed. For problem 3, I have created a program where the code will show how the searching works for binary, it will show the steps until they have found the matched value from the array. We can also notice that we are always using sorted lists for binary search, because it will compare a number to a new value.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have used an AI to generate the sense and concepts of the tasks.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.